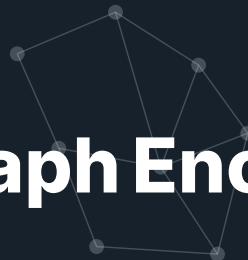


RESEARCH + ENGINEERING SPECIFICATION

Biologically Grounded Graph Encoder for Perturbation Identity



Comprehensive Research and Engineering Design Specification

VERSION	0.1
STATUS	Comprehensive design specification
PRIMARY USE CASE	Unseen-gene perturbation response prediction in K562
TARGET OUTPUT	A learned perturbation identity embedding consumed by the student query pathway
BASELINE	Static PPI graph + rich node features + one-hop mean message passing
COMPARISON	Legacy grouped-ridge identity and AdaPert-style adaptive subgraph selection

July 2026

Perturbation Representation Research Program

Document Purpose

This specification defines the scientific rationale, mathematical formulation, data contracts, graph construction procedures, topology diagnostics, implementation architecture, experimental program, and version roadmap for a biologically grounded perturbation identity encoder. It is intended to function simultaneously as a research blueprint, an engineering contract, and the foundation of a future methods section.

Design stance

The first implementation remains deliberately plain: rich gene descriptors, one binary PPI graph, one mean-aggregation layer, target-node readout, and end-to-end student supervision. More ambitious mechanisms must earn their place through topology and prediction evidence.

Contents

Document Purpose	i
0. Executive Summary	1
Part I. Research Vision	2
1. Problem Statement	2
2. Core Hypothesis	2
3. Architectural Invariants	3
3.1 Node descriptors and topology remain separate	3
3.2 Every modality receives an independent encoder before fusion	4
3.3 The teacher remains privileged	4
3.4 One major modeling assumption changes per version	4
3.5 Every added mechanism requires an ablation	4
3.6 Train, tune, and hold perturbations remain strictly separated	4
Part II. Existing System and Integration Boundary	5
4. Existing Perturbation Identity Pathway	5
5. Graph Encoder Integration	5
6. Other Consumers of the Shared Graph Embedding	6
6.1 Student query head	6
6.2 Control coefficient head	6
6.3 Teacher	7
Part III. Data Universes and Identifiers	8
7. Distinct Gene Universes	8
7.1 Perturbation universe	8
7.2 Response universe	8
7.3 Graph context universe	8
8. Why the Graph Universe May Exceed the Response Universe	9
9. Identifier Canonicalization	9
Part IV. Node Feature Construction	11
10. V1 AlphaGenome Modalities	11
11. Gene-Level Aggregation of Track Predictions	11
12. Additional Node Descriptor Families	12
13. Per-Modality Encoders	13
14. Fusion Encoder	13
Part V. Graph Priors	15
15. Mechanistically Distinct Graph Families	15
15.1 PPI	15

15.2 Gene regulatory network	15
15.3 Signaling network	15
15.4 Functional or semantic relation	16
16. V1 PPI Graph	16
17. Graph Context Construction Options	17
17.1 Full available graph	17
17.2 Perturbation-centered one-hop universe	17
17.3 Response-gene universe	17
Part VI. V1 Message Passing	18
18. Mean-Aggregation GraphSAGE Baseline	18
19. One-Hop Interpretation	18
20. V1 Output	19
Part VII. Exact AdaPert Reconstruction and Contrast	20
21. AdaPert Fixed Graph Construction	20
22. AdaPert Structural Node Encoder	20
23. AdaPert Semantic Perturbation Embedding	21
24. AdaPert Node Scoring	21
25. AdaPert Differentiable Node Sampling	22
26. AdaPert Versus Proposed Encoder	22
Part VIII. Topology Oracle Analysis	24
27. Purpose	24
28. Ground-Truth Perturbation Effect	24
29. Response Mass	25
30. Hop Distance	25
31. Response-Mass Coverage at Hop Limit L	26
32. Coverage and Quality Must Be Reported Separately	26
33. Top- K DEG Coverage Variant	27
34. Density Confounding	27
34.1 Matched edge budget	28
34.2 Pareto frontier	28
Part IX. Multi-View Graph Construction	29
35. Candidate Edge Union	29
36. AlphaGenome Similarity as a Construction Heuristic	29
37. Duplicate Edges Across Priors	30
38. Sweep Over K	30
39. K and L Are Different	31
Part X. Version Roadmap	32
40. V1: Static Single-Relation Graph Encoder	32
Node features	32
Edges	32
Message passing	32
Output	32

Training	32
Key experiments	32
41. V2: Fixed Multi-Relation Graph	33
42. V3: Learn Edge Importance or Refine Structure	33
43. V4: Perturbation-Conditioned Subgraph	34
Part XI. Training Objectives	35
44. Student Prediction Loss	35
45. Correlation Loss	35
46. Gradient Flow	35
47. Multi-Task Control Head	36
Part XII. Experimental Program	37
48. Baseline Audit	37
48.1 Legacy baseline	37
48.2 Corrected legacy baseline	37
48.3 Fusion-only baseline	37
49. Core V1 Ablations	37
50. Node-Universe Ablation	38
51. Prior Graph Bake-Off	38
52. Split Discipline	39
Part XIII. Evaluation Metrics	40
53. Prediction Metrics	40
54. Graph Metrics	40
55. Representation Diagnostics	40
Part XIV. Failure Modes	42
56. Graph Oversmoothing	42
57. High-Degree Dominance	42
58. Missing Perturbation Nodes	42
59. Feature Leakage or Circularity	43
60. Graph Density Illusion	43
61. Feature Dominance	43
62. Misinterpreting AlphaGenome Similarity	43
Part XV. Software Architecture	44
63. Semantic Layer	44
64. Tensorization Layer	44
65. Suggested Modules	45
66. Graph Encoder Interface	45
Part XVI. Provenance	47
67. Three Provenance Objects	47
67.1 ArtifactSpec	47
67.2 ResolvedArtifactSpec	47
67.3 Artifact	47
68. Required Graph Artifact Provenance	48

Part XVII. Decision Log	49
Part XVIII. Open Questions	50
Part XIX. Immediate Execution Plan	51
Phase 0: Audit and freeze	51
Phase 1: Topology diagnostics	51
Phase 2: V1 feature pipeline	51
Phase 3: V1 graph encoder	51
Phase 4: Graph construction study	51
Phase 5: Later versions	52
Part XX. Acceptance Criteria for V1	53
Closing Position	54

0. Executive Summary

This document specifies a staged research program for replacing a static, precomputed perturbation identity prior with a learned graph encoder.

The proposed encoder represents every gene as a node with biologically meaningful node descriptors. In the initial version, those descriptors are derived from four AlphaGenome-predicted functional-genomics modalities:

1. RNA-seq
2. TF ChIP-seq
3. Histone ChIP-seq
4. ATAC-seq

A biological graph supplies edges. The first graph is a binary, undirected protein-protein interaction graph. A one-layer mean-aggregation graph encoder updates every node by combining its own descriptor with the mean descriptor of its neighbors. The rows corresponding to the 2,057 target perturbations are then extracted and used as perturbation identity embeddings.

The model is trained end-to-end through the existing student objective. The teacher pathway remains unchanged in the first experiment.

The larger research program separates three questions:

1. **Graph topology quality:** Does a candidate biological graph place true response genes near the perturbed gene?
2. **Graph construction quality:** Can a sparse multi-view graph preserve or improve biologically relevant reachability?
3. **Representation learning quality:** Does message passing over that graph improve perturbation prediction?

A topology-only oracle analysis is introduced to answer the first two questions without training a neural model. For every perturbation and hop limit L , it measures the fraction of observed response magnitude reachable within L graph hops. This creates a reusable diagnostic for PPI, GRN, signaling, pathway, union, pruned, and eventually learned graphs.

The roadmap is:

- **V1:** One static graph, rich node features, one-hop mean aggregation.
 - **V2:** Fixed heterogeneous or multi-view relations.
 - **V3:** Learn edge weights or refine graph structure.
 - **V4:** Learn a perturbation-conditioned node set and induced subgraph.
-

Part I. Research Vision

1. Problem Statement

Current perturbation models often treat perturbation identity as a fixed vector assembled before model training. In the present system, heterogeneous biological priors are concatenated and compressed using grouped ridge regression. That embedding is then treated as an immutable input to the downstream model.

This design has several limitations:

- the priors are fused outside the final prediction objective;
- the representation cannot adapt to the downstream task;
- interactions among modalities are mostly linear;
- biological relationships are encoded as tabular features rather than topology;
- implementation defects in identifier resolution or prior ingestion can silently corrupt the embedding;
- the representation does not clearly distinguish intrinsic properties of a gene from its biological context.

The proposed encoder reframes perturbation identity as a learned object.

A perturbation is not merely a row in a prior matrix. It is a gene node embedded in a biological network, with intrinsic molecular descriptors and a neighborhood of biologically related genes.

The encoder must therefore answer two questions:

1. What do we know about the gene itself?
2. What biological context surrounds it?

Node features answer the first question. Graph edges answer the second.

2. Core Hypothesis

Let p denote a perturbed gene. Its useful representation should depend on both its intrinsic descriptors and the descriptors of biologically related genes:

$$z_p = f_{\theta}(x_p, \{x_u : u \in \mathcal{N}(p)\}, \mathcal{G}),$$

where:

- x_p is the initial descriptor for perturbation gene p ;
- $\mathcal{N}(p)$ is the set of graph neighbors;
- $\mathcal{G}$ is the biological graph;
- $z_p \in \mathbb{R}^d$ is the learned perturbation identity embedding.

The principal hypothesis is:

A task-trained graph representation of perturbation identity will generalize better to unseen perturbations than a frozen, independently trained ridge embedding.

A second hypothesis is:

Different biological graphs encode distinct mechanisms, and their usefulness varies across perturbation classes.

For example:

- transcription-factor perturbations may be better organized by GRNs;
- kinase perturbations may be better organized by signaling or PPI graphs;
- structural-complex perturbations may be strongly represented by PPI;
- broad functional similarity may be captured by GO or pathway structure.

A third hypothesis is:

A graph should be evaluated before it is used in a GNN.

The topology-only oracle analysis defined later tests whether true experimental responses are reachable from perturbation nodes in the proposed prior graph.

3. Architectural Invariants

3.1 Node descriptors and topology remain separate

Node descriptors answer:

What is this gene?

Edges answer:

How is this gene related to another gene?

Do not encode adjacency inside the node feature matrix.

3.2 Every modality receives an independent encoder before fusion

AlphaGenome modalities differ in scale, sparsity, interpretation, and dimensionality. They should not be concatenated raw and forced through one shared first layer.

3.3 The teacher remains privileged

The teacher may use observed response-derived information that is unavailable at inference. The graph encoder is intended for the student and other inference-time components.

3.4 One major modeling assumption changes per version

- V1 changes the perturbation identity encoder.
- V2 changes relation structure.
- V3 changes graph structure learning.
- V4 changes the graph from global to perturbation-conditioned.

3.5 Every added mechanism requires an ablation

No new graph, modality, edge weight, attention mechanism, or adaptive selector is accepted without an isolated comparison.

3.6 Train, tune, and hold perturbations remain strictly separated

Any statistic that uses observed treated responses must be computed on the fit partition only unless it is used solely for final evaluation.

Part II. Existing System and Integration Boundary

4. Existing Perturbation Identity Pathway

The current perturbation identity pipeline is approximately:

```
Biological prior blocks
    |
Concatenate prior vectors
    |
Grouped ridge regression on fit responses
    |
Frozen perturbation identity vector
    |
Student query head
    |
Retrieval / donor weighting / response prediction
```

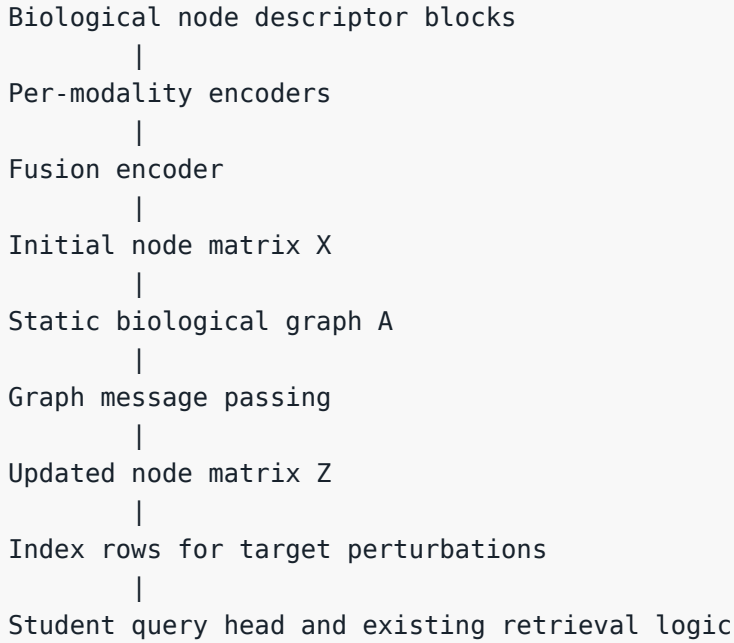
The current retrieval system also uses the perturbation identity embedding in a manually weighted similarity term. Therefore, replacing the embedding is not isolated to one internal tensor consumer. The graph embedding must replace the legacy embedding everywhere the student-side identity is consumed.

For the first graph experiment:

- leave the teacher representation unchanged;
- leave the retrieval formula unchanged;
- replace the legacy perturbation identity embedding everywhere on the student side;
- clearly report that the evaluated object is the **student perturbation identity subsystem**, not only one encoder layer.

5. Graph Encoder Integration

The proposed path is:



The graph encoder returns one vector for each graph node:

$$Z = \text{GraphEncoder}_{\theta}(X, A) \in \mathbb{R}^{N \times d}.$$

Let $\mathcal{P}$ be the ordered set of target perturbation genes. The perturbation embedding matrix is obtained by row gathering:

$$Z_{\mathcal{P}} = Z[I_{\mathcal{P}}, :] \in \mathbb{R}^{|\mathcal{P}| \times d},$$

where $I_{\mathcal{P}}$ maps target gene identifiers to graph row indices.

No pooling over the full graph is required in V1. Each perturbation is represented by its updated node row.

6. Other Consumers of the Shared Graph Embedding

6.1 Student query head

The graph embedding replaces the current grouped-ridge identity vector.

6.2 Control coefficient head

The existing predictor for the 19-dimensional control-state coefficient vector may consume the same graph embedding through its existing MLP. This is a secondary experiment, not required for the cleanest first student-only test.

6.3 Teacher

The teacher remains unchanged in V1 because it consumes response-derived and control-derived privileged information.

Part III. Data Universes and Identifiers

7. Distinct Gene Universes

Three sets must not be conflated.

7.1 Perturbation universe

$$\mathcal{P} = \{p_1, \dots, p_M\}, \quad M \approx 2,057.$$

These are the perturbation targets for which the model must produce identity embeddings and predictions.

7.2 Response universe

$$\mathcal{R} = \{r_1, \dots, r_D\}, \quad D = 5,000.$$

These are the genes in the scored response vector.

The output target for one perturbation is:

$$y_p \in \mathbb{R}^D.$$

The response universe does not need to equal the graph node universe.

7.3 Graph context universe

$$\mathcal{V} = \{v_1, \dots, v_N\}, \quad N \approx 11,000.$$

This is the set of genes included in message passing.

A candidate construction is:

$$\mathcal{V} = \mathcal{P} \cup \bigcup_{p \in \mathcal{P}} \mathcal{N}_{\mathcal{G}}^{(1)}(p),$$

where $\mathcal{N}_{\mathcal{G}}^{(1)}(p)$ is the one-hop neighborhood of perturbation p in the chosen prior graph.

This universe preserves direct graph context around every perturbation while allowing unmeasured mediator genes to influence the perturbation embedding.

8. Why the Graph Universe May Exceed the Response Universe

The model predicts expression only for the 5,000 response genes, but a causal or regulatory path can traverse genes outside that scored set.

A mediator may:

- change activity without showing a large expression change;
- be outside the highly variable gene panel;
- transmit a signal by phosphorylation;
- activate a transcription factor;
- participate in a protein complex;
- alter chromatin state;
- connect the perturbed gene to downstream DEGs.

Restricting graph nodes to response genes creates a measurement-centric inductive bias:

Only genes we score are allowed to mediate the perturbation.

Using a broader context graph creates an intervention-centric inductive bias:

Genes with plausible mechanistic proximity to perturbations may contribute even when they are not output dimensions.

This is a foundational design choice and must be analyzed empirically.

9. Identifier Canonicalization

Every graph, node feature block, perturbation label, and response matrix must resolve to one canonical gene identity table.

Recommended canonical fields:

```
gene_id:
  hgnc_id: "HGNC:11998"
  symbol: "TP53"
  ensembl_gene_id: "ENSG00000141510"
  entrez_gene_id: "7157"
  biotype: "protein_coding"
  aliases:
    - "P53"
    - "BCC7"
```

Required validation:

- no duplicate canonical genes;
- no alias collision;
- explicit pseudogene handling;
- protein-to-gene mapping is one-to-many aware;
- transcript identifiers are not silently treated as gene identifiers;
- missing modalities are accompanied by masks;
- perturbation labels are checked against the canonical table before any split.

The corrected legacy ridge baseline should be generated only after this audit.

Part IV. Node Feature Construction

10. V1 AlphaGenome Modalities

The initial AlphaGenome node descriptor uses four output families:

1. **RNA-seq**
Predicted transcript abundance or coverage.
2. **TF ChIP-seq**
Predicted binding signal for specific transcription factors.
3. **Histone ChIP-seq**
Predicted histone-mark signals reflecting regulatory chromatin states.
4. **ATAC-seq**
Predicted chromatin accessibility.

These modalities are complementary:

- ATAC-seq: whether chromatin is open;
- TF ChIP-seq: whether a specific factor is predicted to bind;
- histone ChIP-seq: regulatory state;
- RNA-seq: transcriptional output.

DNase-seq is initially excluded because it overlaps strongly with ATAC-seq in biological question. CAGE, PRO-cap, splicing, and contact maps are deferred.

11. Gene-Level Aggregation of Track Predictions

AlphaGenome produces spatial tracks, not automatically one vector per gene. A reproducible aggregation rule is therefore required.

For each gene i and modality m , define a raw feature block:

$$x_i^{(m)} \in \mathbb{R}^{d_m}.$$

Possible aggregation regions:

- promoter: TSS $\pm$ 2 kb;
- proximal regulatory window: TSS $\pm$ 10 kb;
- gene body;

- full AlphaGenome sequence window;
- annotated enhancers linked to the gene;
- fixed bins relative to TSS.

Possible summary statistics:

- mean;
- maximum;
- area under predicted track;
- quantiles;
- strand-aware sums;
- peak count;
- top- q peak magnitudes;
- learned pooling over bins.

V1 should use one deterministic, documented aggregation per modality. A recommended starting point is:

- RNA-seq: gene-body and exon-aware summary;
- ATAC-seq: promoter and proximal-window peak summaries;
- TF ChIP-seq: promoter/proximal summaries across available factors;
- histone ChIP-seq: promoter/proximal summaries across marks.

Every modality must include:

- raw dimension;
- aggregation window;
- transformation;
- normalization;
- missingness mask;
- versioned metadata describing AlphaGenome request settings.

12. Additional Node Descriptor Families

The earlier design also considered:

- ESM protein sequence embeddings;
- GO structured features or embeddings.

These remain valid additions, but the four AlphaGenome blocks alone form the narrowest experimental version. A clean sequence of node-feature ablations is:

1. AlphaGenome only;
2. ESM only;
3. GO only;
4. AlphaGenome + ESM;

5. AlphaGenome + GO;
6. AlphaGenome + ESM + GO.

13. Per-Modality Encoders

Each raw block receives an independent encoder:

$$h_i^{(m)} = f_{\theta_m}(x_i^{(m)}), \quad h_i^{(m)} \in \mathbb{R}^{q_m}.$$

A baseline MLP:

$$f_{\theta_m}(x) = W_2^{(m)} \phi(\text{LN}(W_1^{(m)} x + b_1^{(m)})) + b_2^{(m)}.$$

Recommended components:

- linear layer;
- GELU;
- LayerNorm;
- dropout;
- second linear layer;
- optional residual connection when dimensions permit.

Do not validate Pydantic domain objects inside this hot path. Validation occurs at ingestion and tensorization boundaries.

14. Fusion Encoder

Concatenate encoded modality blocks:

$$h_i = h_i^{(\text{RNA})} \parallel h_i^{(\text{TF})} \parallel h_i^{(\text{Histone})} \parallel h_i^{(\text{ATAC})}.$$

Then project to the graph hidden dimension:

$$x_i^{(0)} = f_{\text{fuse}}(h_i) \in \mathbb{R}^d.$$

For V1:

$$d = 128.$$

Stacking all nodes gives:

$$X^{(0)} = \begin{bmatrix} (x_1^{(0)})^\top \\ \vdots \\ (x_N^{(0)})^\top \end{bmatrix} \in \mathbb{R}^{N \times 128}.$$

Part V. Graph Priors

15. Mechanistically Distinct Graph Families

Candidate graph priors should be categorized by edge semantics rather than database name.

15.1 PPI

Edge meaning:

The protein products physically associate or participate in a reported protein-protein relationship.

Often undirected and context-agnostic.

Example resources:

- STRING;
- BioGRID;
- IntAct.

15.2 Gene regulatory network

Edge meaning:

The product of gene u , typically a transcription factor, regulates transcription of gene v .

Usually directed and sometimes signed.

Possible evidence:

- TF ChIP-seq;
- motif presence;
- accessibility;
- expression perturbation;
- curated TF-target relationships.

15.3 Signaling network

Edge meaning:

Protein u activates, inhibits, phosphorylates, dephosphorylates, or otherwise transmits a directional biochemical signal to protein v .

Usually directed and signed or typed.

Example resources:

- OmniPath;
- SIGNOR;
- Reactome reaction relations;
- kinase-substrate databases.

15.4 Functional or semantic relation

Edge meaning:

Genes are functionally similar or share ontology/pathway membership.

Examples:

- GO semantic similarity;
- pathway co-membership;
- co-expression.

This is less mechanistically causal than PPI, GRN, or signaling.

16. V1 PPI Graph

The first graph is:

- one node type: gene;
- one edge type: PPI;
- undirected;
- binary;
- no edge weights;
- no direction;
- no attention;
- no learned structure.

Let:

$$A \in \{0, 1\}^{N \times N}$$

with:

$$A_{ij} = 1 \iff (i, j) \in E_{\text{PPI}}.$$

The matrix is conceptually $N \times N$, but stored sparsely as:

- COO edge_index;
- CSR;
- sparse tensor.

Never materialize a dense $11,000 \times 11,000$ matrix unless required for a diagnostic.

17. Graph Context Construction Options

17.1 Full available graph

Use all genes with features and all retained edges.

Advantage: simplest and does not require an arbitrary neighborhood extraction rule.

Disadvantage: may include large irrelevant regions and is difficult to compare across priors with different coverage.

17.2 Perturbation-centered one-hop universe

Use:

$$\mathcal{V} = \mathcal{P} \cup \bigcup_{p \in \mathcal{P}} \mathcal{N}^{(1)}(p).$$

Advantage: guarantees context around target perturbations and preserves potential mediator genes.

Disadvantage: biases the node universe toward known neighbors of benchmark perturbations.

17.3 Response-gene universe

Use only the 5,000 scored response genes.

Advantage: aligns graph nodes with prediction dimensions and is directly comparable to AdaPert's HVG-filtered graph.

Disadvantage: excludes unmeasured mediators and may omit perturbation genes.

These must be treated as different graph-construction philosophies, not minor hyperparameter settings.

Part VI. V1 Message Passing

18. Mean-Aggregation GraphSAGE Baseline

For node i , define its neighborhood:

$$\mathcal{N}(i) = \{j : A_{ij} = 1\}.$$

Neighbor message:

$$m_i^{(0)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} x_j^{(0)}.$$

If a node is isolated, use one of:

- $m_i^{(0)} = 0$;
- add a self-loop;
- use a learned isolated-node vector.

The baseline update is:

$$x_i^{(1)} = \phi \left(W \left[x_i^{(0)} \parallel m_i^{(0)} \right] + b \right),$$

where:

$$W \in \mathbb{R}^{128 \times 256}.$$

The updated matrix is:

$$X^{(1)} \in \mathbb{R}^{N \times 128}.$$

Use one message-passing layer in the first implementation.

19. One-Hop Interpretation

With one layer, perturbation p can receive information only from direct graph neighbors:

$$x_p^{(1)} = f \left(x_p^{(0)}, \{x_j^{(0)} : j \in \mathcal{N}(p)\} \right).$$

A second layer permits two-hop influence, but also creates:

- greater smoothing;
- more dependence on graph density;
- more opportunities for irrelevant signals to enter;
- a harder interpretation.

The topology oracle analysis should guide whether deeper propagation is biologically justified.

20. V1 Output

After message passing:

$$z_p = X^{(1)}[I(p), :] \in \mathbb{R}^{128}.$$

For all perturbations:

$$Z_{\mathcal{P}} \in \mathbb{R}^{2057 \times 128}.$$

This matrix replaces the legacy perturbation identity prior wherever the student consumes it.

Part VII. Exact AdaPert Reconstruction and Contrast

21. AdaPert Fixed Graph Construction

AdaPert uses STRING v11.5 as its graph source.

Its graph-building pipeline is:

1. Download or load the full STRING graph.
2. Map STRING protein entities to genes.
3. Restrict nodes to the highly variable genes used by the experiment.
4. Rank each node's incident edges by STRING confidence.
5. Retain the top k edges per node.
6. Evaluate $k \in \{10, 20\}$.

The paper reports:

- raw graph: 18,382 nodes and 11,257,696 edges;
- HVG-filtered graph: 4,509 nodes and 1,090,554 edges;
- top-20 graph: 89,793 edges;
- top-10 graph: 45,013 edges.

The top- k operation is a ranking rule, not a global confidence threshold.

22. AdaPert Structural Node Encoder

AdaPert initializes each graph node with a one-hot identity vector:

$$h_v^{(0)} = x_v,$$

where x_v is one-hot over the graph node vocabulary.

It then applies neighbor-mean message passing:

$$h_v^{(\ell+1)} = \sum_{u \in \mathcal{N}(v)} \frac{1}{|\mathcal{N}(v)|} W^{(\ell)} h_u^{(\ell)}.$$

Important details:

- the paper does not explicitly concatenate self-state and neighbor state;
- the written equation is simpler than standard GraphSAGE;
- edge confidence is used for top- k graph construction, but not shown in the message equation;
- the node embedding is topology-derived;
- the exact number of layers L is not specified in the main method text.

After L layers:

$$h_v = h_v^{(L)} \in \mathbb{R}^{d_s}.$$

These are structural embeddings, not functional-genomics features.

23. AdaPert Semantic Perturbation Embedding

For perturbation gene p , AdaPert retrieves a textual gene description and embeds it:

$$s_p = \text{LM}(\text{desc}(p)) \in \mathbb{R}^{d_t}.$$

The main text refers to GPT-4o. The appendix identifies GenePT embeddings from NCBI and UniProt gene descriptions generated with OpenAI text embedding models, with 1,536- and 3,072-dimensional variants.

The semantic vector is projected to structural dimension:

$$\tilde{s}_p = W_s s_p \in \mathbb{R}^{d_s}.$$

This semantic vector conditions selection. It is not the initial node feature for every graph node.

24. AdaPert Node Scoring

For every graph node v , concatenate:

$$c_v = h_v \parallel \tilde{s}_p.$$

Compute a perturbation-conditioned relevance score:

$$a_v = w^\top \sigma(W_c c_v).$$

Normalize over all nodes:

$$\alpha_v = \frac{\exp(a_v)}{\sum_{u \in V} \exp(a_u)}.$$

25. AdaPert Differentiable Node Sampling

AdaPert uses a Gumbel-Softmax relaxation:

$$\tilde{\alpha}_v = \frac{\exp((\log \alpha_v + g_v)/\tau)}{\sum_{u \in V} \exp((\log \alpha_u + g_u)/\tau)},$$

where:

$$g_v \sim \text{Gumbel}(0, 1).$$

Nodes satisfying:

$$\tilde{\alpha}_v > T$$

are selected.

The selected node set induces a perturbation-specific subgraph:

$$G_p = G[V_p].$$

The perturbation context representation is a sum of selected structural node embeddings:

$$z_{\text{context}}^{(p)} = \sum_{v \in V(G_p)} h_v.$$

The paper does not define an additional MLP after this sum in the stated context equation.

26. AdaPert Versus Proposed Encoder

Dimension	AdaPert	Proposed V1
Node universe	HVGs used in experiment	Perturbations plus broader context genes
Static graph	STRING PPI, top- k	Binary PPI
Initial node features	One-hot identity	AlphaGenome feature fusion
Structural encoder	Neighbor-mean topology encoder	GraphSAGE-style self + neighbor update
Perturbation conditioning	Text semantic embedding	Perturbation's own graph node row
Adaptive selection	Gumbel-Softmax node selection	None

Dimension	AdaPert	Proposed V1
Final perturbation representation	Sum over selected structural nodes	Updated target-node embedding
Main inductive bias	Sparse perturbation-specific context	Rich local biological context

AdaPert asks:

Which nodes in a global graph are relevant to this perturbation?

The proposed V1 asks:

How should the perturbation node’s own biological descriptor be updated using its local biological context?

Part VIII. Topology Oracle Analysis

27. Purpose

Before training a GNN, determine whether a graph prior places observed response genes near the perturbed gene.

This analysis uses:

- perturbation identities;
- observed perturbation effects;
- graph topology.

It does not use:

- AlphaGenome node values;
- learned message functions;
- prediction model parameters.

This separates graph quality from model quality.

28. Ground-Truth Perturbation Effect

For perturbation p , define the observed effect vector:

$$\Delta y_p = y_p^{\text{treated}} - y_p^{\text{control}} \in \mathbb{R}^D.$$

Possible summary targets:

- mean effect across cells;
- OT-matched control-to-response effect;
- log-fold change;
- standardized effect;
- signed or absolute effect.

The oracle analysis should use the same fit-only effect definition that is used elsewhere in training diagnostics.

29. Response Mass

Define total response mass:

$$M_p = \sum_{g \in \mathcal{R}} w_{p,g},$$

where a basic choice is:

$$w_{p,g} = |\Delta y_{p,g}|.$$

Alternative weights:

- squared effect;
- absolute standardized effect;
- DEG significance-weighted effect;
- top- K -truncated effect;
- rank-decayed effect.

The absolute-effect version is preferred initially because it preserves broad response structure without requiring an arbitrary DEG cutoff.

30. Hop Distance

For graph G , define:

$$d_G(p, g)$$

as the shortest-path hop distance from perturbation node p to response gene g .

If either node is absent or no path exists:

$$d_G(p, g) = \infty.$$

Compute distances with:

- BFS for unweighted graphs;
 - batched BFS;
 - directed shortest paths for directed GRNs/signaling graphs;
 - undirected sensitivity analysis where direction may be incomplete.
-

31. Response-Mass Coverage at Hop Limit L

For perturbation p :

$$C_p^G(L) = \frac{\sum_{g \in \mathcal{R}} w_{p,g} \mathbf{1}[d_G(p, g) \leq L]}{\sum_{g \in \mathcal{R}} w_{p,g}}.$$

Interpretation:

- $C_p^G(1) = 0.70$: 70% of observed response magnitude lies on direct graph neighbors;
- $C_p^G(2) = 0.90$: 90% lies within two hops.

Aggregate:

$$\bar{C}^G(L) = \frac{1}{|\mathcal{P}_G|} \sum_{p \in \mathcal{P}_G} C_p^G(L),$$

where $\mathcal{P}_G$ contains perturbations represented in graph G .

Report:

- mean;
- median;
- quartiles;
- fraction reaching 50%, 80%, 90%, and 95% mass by each L ;
- per-perturbation curves;
- perturbation-class stratification.

32. Coverage and Quality Must Be Reported Separately

A graph that covers only 40% of perturbations may have excellent hop coverage on those it contains. Do not collapse these into one number.

Report:

$$\text{NodeCoverage}(G) = \frac{|\mathcal{P}_G|}{|\mathcal{P}|}.$$

Then separately report:

$$\bar{C}^G(L).$$

A useful summary table:

Graph	Perturbation coverage	Avg. degree	Edge count	Mass @1 hop	Mass @2 hops	Mass @3 hops
PPI						
GRN						
Signaling						
Union						

33. Top- K DEG Coverage Variant

A DEG-ranked diagnostic can supplement response-mass coverage.

Let:

$$\text{TopDEG}_p(K)$$

be the top K response genes ranked by $|\Delta y_{p,g}|$ or a fit-only differential-expression statistic.

Define:

$$R_p^G(L, K) = \frac{|\{g \in \text{TopDEG}_p(K) : d_G(p, g) \leq L\}|}{K}.$$

Use several values:

$$K \in \{20, 50, 100, 200, 500\}.$$

Do not select K using hold data.

34. Density Confounding

Dense graphs reduce shortest-path distances almost automatically.

Every oracle result must be accompanied by graph complexity:

- edge count;
- average degree;
- median degree;
- largest connected component;
- diameter or effective diameter;
- isolated node count.

Two fair comparison strategies:

34.1 Matched edge budget

Prune each graph to a comparable edge count or mean degree.

34.2 Pareto frontier

Plot:

- x-axis: average degree or edge count;
- y-axis: response mass within a fixed hop limit.

A graph is preferable if it achieves more response coverage at the same or lower density.

Part IX. Multi-View Graph Construction

35. Candidate Edge Union

Let relation-specific edge sets be:

$$E_{\text{PPI}}, \quad E_{\text{GRN}}, \quad E_{\text{SIG}}, \quad E_{\text{FUNC}}.$$

Construct candidate union:

$$E_{\text{cand}} = E_{\text{PPI}} \cup E_{\text{GRN}} \cup E_{\text{SIG}} \cup E_{\text{FUNC}}.$$

Collapse duplicate node pairs into one candidate edge while retaining provenance metadata.

For each candidate edge:

```
source_gene: TP53
target_gene: CDKN1A
supported_by:
  - GRN
  - Reactome
directed_support:
  GRN: true
  Reactome: true
```

The first global fused graph may ignore edge type during message passing, but provenance should not be discarded from storage.

36. AlphaGenome Similarity as a Construction Heuristic

Let q_i be a fixed AlphaGenome descriptor used only for graph construction.

For candidate edge (i, j) , define:

$$s_{ij}^{\text{AG}} = \cos(q_i, q_j) = \frac{q_i^\top q_j}{\|q_i\|_2 \|q_j\|_2}.$$

For each source node i :

1. collect all unique candidate neighbors;
2. compute s_{ij}^{AG} ;
3. rank neighbors;
4. retain the top K unique neighbors.

This yields:

$$E_K^{\text{AG}}.$$

Important separation:

- AlphaGenome similarity constructs the graph;
- the oracle hop metric evaluates the graph;
- the GNN later learns node representations on the graph.

The topology oracle does not use AlphaGenome features during evaluation.

37. Duplicate Edges Across Priors

If multiple priors propose the same neighbor:

- collapse to one edge;
- retain source provenance;
- do not count the neighbor multiple times in mean aggregation;
- do not allow duplicate rows in the top- K list.

V1 fused-graph score:

$$s_{ij} = s_{ij}^{\text{AG}}.$$

Possible later score:

$$s_{ij} = \alpha s_{ij}^{\text{AG}} + \beta s_{ij}^{\text{prior}} + \gamma a_{ij}^{\text{agreement}}.$$

Do not introduce this composite score until plain cosine pruning is evaluated.

38. Sweep Over K

K is a graph-construction parameter.

For:

$$K \in \{5, 10, 20, 40, 80, 160\},$$

build a graph and report:

- edge count;
- mean degree;
- connected components;
- response mass at $L = 1, 2, 3$;
- top-DEG recall at $L = 1, 2, 3$;
- perturbation coverage.

Select the smallest K that preserves a predeclared fraction of the best attainable topology score.

Example rule:

Choose the smallest K whose two-hop response-mass coverage is at least 95% of the maximum coverage across the sweep.

This must be selected on fit/tune only.

39. K and L Are Different

- K : number of retained neighbors per node during graph construction.
- L : number of graph hops allowed during topology evaluation or message passing.

The workflow is:

1. choose or sweep K ;
2. construct graph;
3. evaluate coverage for multiple L ;
4. choose message-passing depth using topology evidence and validation performance.

Do not infer $K = L$.

Part X. Version Roadmap

40. V1: Static Single-Relation Graph Encoder

Node features

- AlphaGenome RNA-seq;
- TF ChIP-seq;
- histone ChIP-seq;
- ATAC-seq.

Edges

- one binary undirected PPI graph.

Message passing

- one mean-aggregation GraphSAGE layer;
- hidden dimension 128.

Output

- gather the 2,057 perturbation rows.

Training

- end-to-end through student-side losses;
- teacher unchanged.

Key experiments

1. corrected grouped-ridge baseline;
2. fusion MLP without graph;
3. graph encoder with PPI;
4. node-universe sensitivity:
 - response genes only;
 - perturbations plus one-hop context;
 - full available context graph.

41. V2: Fixed Multi-Relation Graph

All nodes remain genes. Edge types differ.

For relation r :

$$m_{i,r}^{(\ell)} = \frac{1}{|\mathcal{N}_r(i)|} \sum_{j \in \mathcal{N}_r(i)} h_j^{(\ell)}.$$

Relation-specific messages are fused:

$$m_i^{(\ell)} = f_{\text{rel}}(m_{i,1}^{(\ell)} \parallel \dots \parallel m_{i,R}^{(\ell)}).$$

Node update:

$$h_i^{(\ell+1)} = f_{\text{node}}(h_i^{(\ell)} \parallel m_i^{(\ell)}).$$

Candidate relations:

- PPI;
- GRN;
- signaling;
- functional.

Topology remains fixed.

42. V3: Learn Edge Importance or Refine Structure

The gentlest V3 learns weights only on existing edges.

For candidate edge (i, j, r) :

$$e_{ijr} = f_{\psi}(h_i, h_j, r, x_{ij}^{\text{prior}}).$$

Normalized neighborhood weights:

$$\alpha_{ijr} = \frac{\exp(e_{ijr})}{\sum_{u \in \mathcal{N}_r(i)} \exp(e_{iur})}.$$

Message:

$$m_{i,r} = \sum_{j \in \mathcal{N}_r(i)} \alpha_{ijr} W_r h_j.$$

This keeps the candidate graph fixed while allowing task-specific importance.

Later V3 extensions may permit sparse new edges, but only within a constrained candidate pool.

43. V4: Perturbation-Conditioned Subgraph

Given perturbation p , select a perturbation-specific node set:

$$V_p = \{v : s_{p,v} > T_p\}.$$

The node count is allowed to vary by perturbation.

The induced subgraph is:

$$G_p = G[V_p].$$

Recommended progression:

1. fixed global graph;
2. learned node relevance;
3. differentiable soft selection;
4. hard induced subgraph at inference.

The selector may use:

- perturbation embedding;
- graph structural embeddings;
- AlphaGenome features;
- relation provenance;
- control-state conditioning.

This is substantially closer to AdaPert, but the proposed system may condition on richer node descriptors and multi-view topology.

Part XI. Training Objectives

44. Student Prediction Loss

Let:

$$\hat{y}_p \in \mathbb{R}^D$$

be the predicted perturbation response.

A baseline coefficient MSE:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|\mathcal{B}|} \sum_{p \in \mathcal{B}} \|\hat{y}_p - y_p\|_2^2.$$

If the current model operates in coefficient space, use the current coefficient target and preserve all downstream decoding.

45. Correlation Loss

A correlation term may supplement MSE:

$$\mathcal{L}_{\text{corr}} = 1 - \text{corr}(\hat{y}_p, y_p).$$

Composite:

$$\mathcal{L}_{\text{student}} = \lambda_{\text{MSE}} \mathcal{L}_{\text{MSE}} + \lambda_{\text{corr}} \mathcal{L}_{\text{corr}}.$$

Keep the same student loss as the legacy model for the initial comparison.

46. Gradient Flow

The graph encoder is not precomputed.

$$\mathcal{C} \rightarrow \text{student head} \rightarrow Z_{\mathcal{P}} \rightarrow \text{graph layer} \rightarrow \text{fusion MLP} \rightarrow \text{modality MLPs}.$$

Thus the four AlphaGenome encoders and graph update are learned for the final task.
If graph topology is fixed, there is no gradient into the adjacency itself.

47. Multi-Task Control Head

A later objective may add:

$$\mathcal{L}_{\text{control}} = \|\hat{c}_p - c_p\|_2^2,$$

where:

$$c_p \in \mathbb{R}^{19}$$

is the control coefficient target.

Composite:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{student}} + \lambda_{\text{control}} \mathcal{L}_{\text{control}}.$$

Do not add this in the cleanest first graph experiment unless already active in the comparable legacy setup.

Part XII. Experimental Program

48. Baseline Audit

Before evaluating the graph encoder, produce three baselines.

48.1 Legacy baseline

Original code and artifacts unchanged.

48.2 Corrected legacy baseline

Fix:

- identifier errors;
- pseudogene handling;
- incorrect protein mappings;
- 18% interaction-consumption bug;
- silent missing-feature behavior.

Do not alter the intended modeling philosophy.

48.3 Fusion-only baseline

Use the same per-modality encoders and fusion MLP, but no graph message passing:

$$z_p = x_p^{(0)}.$$

This isolates the value of graph topology.

49. Core V1 Ablations

ID	Node features	Graph	Message passing
A0	Legacy priors	None	Grouped ridge
A1	Corrected legacy priors	None	Grouped ridge
A2	AlphaGenome	None	Fusion MLP
A3	AlphaGenome	PPI	One-hop mean

ID	Node features	Graph	Message passing
A4	AlphaGenome	PPI	Two-hop mean
A5	AlphaGenome + ESM	PPI	One-hop mean
A6	AlphaGenome + GO	PPI	One-hop mean

50. Node-Universe Ablation

Use identical features and graph source.

1. response-gene universe;
2. perturbation-plus-one-hop universe;
3. full feature-covered graph universe.

Report:

- graph size;
- perturbation coverage;
- oracle topology;
- prediction metrics;
- runtime;
- isolated perturbations.

This is a structural ablation, not a minor hyperparameter test.

51. Prior Graph Bake-Off

Evaluate topology-only first:

- STRING PPI;
- GRN;
- signaling;
- functional graph;
- raw union;
- AlphaGenome-pruned union.

Then train the same graph encoder on the strongest candidates.

52. Split Discipline

Graph topology and external node features may be built for all genes if they do not use treated response labels.

However:

- top- K DEG sets;
- response-mass weights;
- learned graph pruning;
- hyperparameter choice;
- relation weighting;

must not use hold perturbation responses.

Use:

- fit perturbations for training;
- tune perturbations for model and graph selection;
- hold perturbations once for final reporting.

The oracle topology analysis can be reported on hold as a final descriptive result only after all choices are frozen.

Part XIII. Evaluation Metrics

53. Prediction Metrics

At minimum:

- coefficient MSE;
- Pearson correlation;
- Pearson- Δ ;
- Spearman- Δ ;
- perturbation discrimination;
- direction match;
- top- K DEG overlap;
- mean-collapse diagnostics.

Report per perturbation and aggregate.

54. Graph Metrics

For every graph:

- node count;
 - edge count;
 - perturbation coverage;
 - response-gene coverage;
 - average degree;
 - median degree;
 - isolated nodes;
 - connected components;
 - degree distribution;
 - relation overlap;
 - response mass at one, two, and three hops;
 - top- K DEG recall at one, two, and three hops.
-

55. Representation Diagnostics

For perturbation embeddings:

- pairwise cosine distribution;
 - norm distribution;
 - collapse score;
 - nearest-neighbor biological enrichment;
 - relation to perturbation class;
 - train-versus-hold separability;
 - retrieval donor composition;
 - sensitivity to graph edge dropout;
 - sensitivity to modality dropout.
-

Part XIV. Failure Modes

56. Graph Oversmoothing

Repeated propagation can make node embeddings similar.

Diagnosis:

- decreasing pairwise distance;
- rank collapse;
- worsening perturbation discrimination;
- little change across perturbation nodes.

Mitigation:

- one layer;
 - residual connections;
 - normalization;
 - jumping knowledge;
 - edge dropout.
-

57. High-Degree Dominance

Hub genes may dominate mean aggregation.

Mitigation:

- degree normalization;
 - top- K pruning;
 - relation-specific messages;
 - hub clipping;
 - attention in later versions.
-

58. Missing Perturbation Nodes

If perturbation $p \notin V$, V_1 cannot gather its row.

Possible policies:

1. expand graph universe to include all perturbations;
2. use fusion-only embedding for missing nodes;
3. attach node through candidate edges;
4. semantic conditioning as in AdaPert.

The policy must be explicit and evaluated.

59. Feature Leakage or Circularity

Do not use treated response data to construct node features for hold perturbations.

AlphaGenome predictions, sequence embeddings, GO, PPI, and curated graphs are external priors. OT-derived treated response statistics are not.

60. Graph Density Illusion

A graph may appear biologically superior only because it is denser.

Always report density and matched-budget analyses.

61. Feature Dominance

RNA-seq may dominate other modalities due to scale or direct relation to the output.

Mitigation:

- per-modality normalization;
 - equal latent dimensions initially;
 - modality dropout;
 - learned gates;
 - ablations.
-

62. Misinterpreting AlphaGenome Similarity

High cosine similarity does not imply causation, regulation, or physical interaction.

Use it only to rank biologically plausible candidate edges, not to create unconstrained causal claims.

Part XV. Software Architecture

63. Semantic Layer

Use Pydantic models at ingestion boundaries.

Example conceptual objects:

```
class GeneIdentity(BaseModel):
    hgnc_id: str
    symbol: str
    ensembl_gene_id: str | None
    entrez_gene_id: str | None
    biotype: str
    aliases: tuple[str, ...] = ()

class FeatureBlockSpec(BaseModel):
    name: str
    source: str
    version: str
    dimension: int
    aggregation: dict[str, object]

class GraphSpec(BaseModel):
    name: str
    source: str
    version: str
    directed: bool
    weighted: bool
    node_universe_artifact: str
    edge_artifact: str
```

These objects are not used inside GPU forward passes.

64. Tensorization Layer

Create immutable training tensors:

```
@dataclass(frozen=True)
class GraphBatch:
    x: torch.Tensor          # [N, D]
    edge_index: torch.Tensor # [2, E]
    node_ids: torch.Tensor   # [N]
    perturbation_rows: torch.Tensor # [M]
```

The model hot path consumes tensors only.

65. Suggested Modules

```
models/
  modality_encoder.py
  modality_fusion.py
  graph_encoder.py
  perturbation_encoder.py
  student_query_head.py
  control_coefficient_head.py

data/
  gene_identity.py
  alphagenome_features.py
  graph_builders.py
  graph_oracle.py
  tensorize.py

experiments/
  baseline_audit.py
  topology_bakeoff.py
  train_v1_graph_encoder.py
  ablate_node_universe.py
```

66. Graph Encoder Interface

```
class PerturbationGraphEncoder(nn.Module):
    def forward(
        self,
        modality_features: dict[str, torch.Tensor],
        edge_index: torch.Tensor,
```

```
    perturbation_rows: torch.Tensor,  
    ) -> torch.Tensor:  
    """  
    Returns:  
        perturbation_embeddings: [M, 128]  
    """
```

The student should not need to know how the perturbation embedding was constructed.

Part XVI. Provenance

67. Three Provenance Objects

67.1 ArtifactSpec

The plan.

Contains:

- intended script;
- input artifact references;
- parameters;
- output location;
- no final execution SHAs.

67.2 ResolvedArtifactSpec

The execution receipt.

Contains:

- SHA of original spec;
- script SHA;
- concrete input artifact SHAs;
- parameters;
- environment;
- timestamps;
- execution status;
- output artifact identifiers.

67.3 Artifact

The concrete output.

Contains:

- path;
- SHA;
- media type;
- producer resolved-spec ID.

This forms a DAG of immutable artifacts and execution receipts.

68. Required Graph Artifact Provenance

Every graph artifact must record:

- source database and version;
- download URL or accession;
- source file checksum;
- identifier mapping artifact;
- node-universe rule;
- edge filtering rule;
- top- K or threshold parameters;
- directed/undirected conversion;
- duplicate collapse policy;
- isolated-node policy;
- final node and edge counts;
- script commit SHA.

Every AlphaGenome feature artifact must record:

- model/API version;
 - reference genome build;
 - sequence interval;
 - requested modalities;
 - ontology terms or biosample tracks;
 - aggregation procedure;
 - missingness handling;
 - raw and processed shapes.
-

Part XVII. Decision Log

Decision	Status	Rationale	Revisit
Hidden dimension 128	Tentative	Matches downstream system	Sweep after V1
Four AlphaGenome modalities	Adopted for narrow V1	Complementary regulatory layers	Add other modalities later
PPI only in V1	Adopted	Isolates graph contribution	V2
Binary edges	Adopted	Clean baseline	Weighted sensitivity
One-hop mean aggregation	Adopted	Stable and interpretable	Topology oracle may justify more
Teacher unchanged	Adopted	Preserves clean comparison	Later joint training
Graph universe $\approx 11k$	Tentative	Includes perturbations and mediators	Node-universe ablation
AlphaGenome-pruned union graph	Planned	Tests multi-view complementarity	After raw prior bake-off
Perturbation-conditioned subgraph	Deferred	Major additional assumption	V4

Part XVIII. Open Questions

1. What exact AlphaGenome aggregation produces one gene-level vector per modality?
 2. Should the graph universe be response-centric, perturbation-centric, or full-coverage?
 3. Which graph prior gives the best density-adjusted oracle coverage?
 4. Does one-hop or two-hop propagation match observed response topology?
 5. Are PPI edges best treated as binary or confidence-weighted?
 6. Should directed graphs remain directed in the oracle analysis?
 7. Does a multi-view AlphaGenome-pruned graph outperform raw priors at matched degree?
 8. Does relation provenance help beyond the collapsed union?
 9. Should the graph encoder also train through the control coefficient head?
 10. How much of current performance depends on the hand-tuned retrieval similarity term?
 11. Can perturbation classes predict which relation type is useful?
 12. Does a learned perturbation-conditioned subgraph improve over the global graph after controlling for density?
-

Part XIX. Immediate Execution Plan

Phase 0: Audit and freeze

1. Canonicalize all gene identities.
2. Reproduce the legacy baseline exactly.
3. Produce corrected ridge baseline.
4. Freeze train/tune/hold perturbation IDs.
5. Freeze response target computation.

Phase 1: Topology diagnostics

1. Build PPI, GRN, and signaling graphs.
2. Compute perturbation and response coverage.
3. Compute response-mass coverage for $L = 1, \dots, L_{\max}$.
4. Compute top- K DEG recall for multiple K .
5. Match graph density where feasible.
6. Produce per-perturbation and aggregate plots.

Phase 2: V1 feature pipeline

1. Validate AlphaGenome feature artifacts.
2. Define gene-level aggregation.
3. Normalize each modality.
4. Train fusion-only baseline.
5. Confirm no collapse or missing-node failures.

Phase 3: V1 graph encoder

1. Build binary PPI graph.
2. Run one-hop mean aggregation.
3. Gather perturbation rows.
4. Replace legacy student identity everywhere.
5. Train under unchanged downstream objective.
6. Compare against corrected ridge and fusion-only.

Phase 4: Graph construction study

1. Union candidate edges from selected priors.

2. Collapse duplicates and retain provenance.
3. Rank unique candidate neighbors by AlphaGenome cosine.
4. Sweep top- K .
5. Apply the same topology oracle.
6. Select graph using tune data and density-aware criteria.

Phase 5: Later versions

1. V2 relation-specific message passing.
 2. V3 learned edge weights.
 3. V4 perturbation-conditioned node selection.
-

Part XX. Acceptance Criteria for V1

V1 is successful if all of the following are true:

1. The graph encoder is trained end-to-end.
 2. All 2,057 required perturbations receive a valid embedding.
 3. No hold response data enter graph construction or node features.
 4. The graph model outperforms fusion-only on at least one primary unseen-perturbation metric.
 5. The graph model is competitive with or better than the corrected ridge baseline.
 6. Embeddings do not collapse.
 7. Results are reproducible from resolved provenance artifacts.
 8. The topology oracle provides a coherent explanation for the chosen graph and message depth.
-

Closing Position

The central scientific contribution is not merely “using a GNN.”

It is the separation and disciplined study of:

1. the gene universe;
2. node biology;
3. graph topology;
4. graph construction;
5. perturbation-conditioned context;
6. end-to-end task supervision.

The topology oracle prevents the project from choosing a graph by intuition alone. The staged roadmap prevents static priors, heterogeneous relations, edge learning, and adaptive subgraphs from being conflated. The corrected legacy baseline prevents implementation bugs from inflating apparent gains.

The first model should remain intentionally plain:

rich gene descriptors, one binary PPI graph, one mean-aggregation layer, target-node read-out, end-to-end student supervision.

Everything more ambitious should earn its place through the same topology and prediction evaluations.